```
    @posts = Post.all
  end
end
```

`Post` is not syntax for a class. Rather, `Post` is a regular Ruby constant. If all is good, the constant is evaluated to an object that responds to `all`.

That is why we talk about *constant* autoloading, Rails has the ability to load constants on the fly.

2.3 Constants are Stored in Modules

Constants belong to modules in a very literal sense. Classes and modules have a constant table; think of it as a hash table.

Let's analyze an example to really understand what that means. While common abuses of language like "the `String` class" are convenient, the exposition is going to be precise here for didactic purposes.

Let's consider the following module definition:

```
module Colors
  RED = '0xff0000'
end
```

First, when the `module` keyword is processed, the interpreter creates a new entry in the constant table of the class object stored in the `Object` constant. Said entry associates the name "Colors" to a newly created module object. Furthermore, the interpreter sets the name of the new module object to be the string "Colors".

Later, when the body of the module definition is interpreted, a new entry is created in the constant table of the module object stored in the `Colors` constant. That entry maps the name "RED" to the string "0xff0000". In particular, `Colors::RED` is totally unrelated to any other `RED` constant that may live in any other class or module object. If there were any, they would have separate entries in their respective constant tables.

Pay special attention in the previous paragraphs to the distinction between class and module objects, constant names, and value objects associated to them in constant tables.

2.4 Resolution Algorithms

2.4.1 Resolution Algorithm for Relative Constants

At any given place in the code, let's define *cref* to be the first element of the nesting if it is not empty, or `Object` otherwise.

Without getting too much into the details, the resolution algorithm for relative constant references goes like this:

1. If the nesting is not empty the constant is looked up in its elements and in order. The ancestors of those elements are ignored.

2. If not found, then the algorithm walks up the ancestor chain of the cref.

3. If not found and the cref is a module, the constant is looked up in `Object`.
4. If not found, `const_missing` is invoked on the cref. The default implementation of `const_missing` raises `NameError`, but it can be overridden.

Rails autoloading **does not emulate this algorithm**, but its starting point is the name of the constant to be autoloaded, and the cref. See more in Relative References.